



PLANTILLA COMPLETA PARA EXAMEN SQL

Procedimientos Almacenados y Triggers



CHECKLIST PRE-EXAMEN



Conceptos Fundamentales

- ☐ Diferencia entre procedimientos y funciones
- ☐ Tipos de triggers (BEFORE/AFTER, INSERT/UPDATE/DELETE)
- ☐ Manejo de transacciones (START TRANSACTION, COMMIT, ROLLBACK)
- ☐ Variables locales vs parámetros
- ☐ Manejo de errores con SIGNAL/RESIGNAL



Sintaxis Básica

- ☐ DELIMITER //
- ☐ Declaración de variables (DECLARE)
- ☐ Estructura IF-THEN-ELSE-END IF
- ☐ Bucles WHILE/LOOP
- ☐ Cursores básicos



PLANTILLAS DE CÓDIGO

1. PROCEDIMIENTO BÁSICO

```
DELIMITER //
```

```
CREATE PROCEDURE nombre_procedimiento(  
    IN parametro_entrada TIPO,  
    OUT parametro_salida TIPO,  
    INOUT parametro_mixto TIPO
```

```
)  
BEGIN  
    -- Declaración de variables locales  
    DECLARE variable_local TIPO DEFAULT valor_defecto;  
    DECLARE contador INT DEFAULT 0;  
  
    -- Lógica del procedimiento  
    SELECT COUNT(*) INTO contador  
    FROM tabla  
    WHERE condicion = parametro_entrada;  
  
    -- Asignar resultado  
    SET parametro_salida = contador;  
  
END //
```

DELIMITER ;

2. PROCEDIMIENTO CON MANEJO DE ERRORES

```
DELIMITER //
```

```
CREATE PROCEDURE procedimiento_con_errores(  
    IN param1 INT,  
    IN param2 VARCHAR(50)  
)  
BEGIN  
    DECLARE v_error_msg VARCHAR(255);  
    DECLARE v_existe INT DEFAULT 0;  
  
    -- Handler para errores SQL  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        GET DIAGNOSTICS CONDITION 1  
            v_error_msg = MESSAGE_TEXT;  
        SELECT CONCAT('ERROR: ', v_error_msg) AS error_message;  
    END;  
  
    -- Iniciar transacción  
    START TRANSACTION;  
  
    -- Validaciones  
    SELECT COUNT(*) INTO v_existe  
    FROM tabla  
    WHERE id = param1;
```

```
IF v_existe = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'El registro no existe';
END IF;

-- Operaciones principales
INSERT INTO tabla VALUES (param1, param2);
UPDATE otra_tabla SET campo = param2 WHERE id = param1;

-- Confirmar transacción
COMMIT;

SELECT 'Operación completada exitosamente' AS resultado;

END //

DELIMITER ;
```

3. PROCEDIMIENTO CON CURSOR

```
DELIMITER //

CREATE PROCEDURE procesar_con_cursor()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_id INT;
    DECLARE v_nombre VARCHAR(100);
    DECLARE v_contador INT DEFAULT 0;

    -- Declarar cursor
    DECLARE cursor_datos CURSOR FOR
        SELECT id, nombre
        FROM tabla
        WHERE condicion = 'valor';

    -- Handler para fin de cursor
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Abrir cursor
    OPEN cursor_datos;

    -- Loop de lectura
    read_loop: LOOP
        FETCH cursor_datos INTO v_id, v_nombre;
```

```
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Procesar cada registro
        UPDATE tabla SET procesado = TRUE WHERE id = v_id;
        SET v_contador = v_contador + 1;

    END LOOP;

    -- Cerrar cursor
    CLOSE cursor_datos;

    SELECT CONCAT('Procesados: ', v_contador, ' registros') AS resultado;

END //
```

DELIMITER ;

4. TRIGGER BEFORE INSERT

```
DELIMITER //
```

```
CREATE TRIGGER trg_before_insert_tabla
    BEFORE INSERT ON tabla
    FOR EACH ROW
BEGIN
    -- Validaciones
    IF NEW.campo < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El campo no puede ser negativo';
    END IF;

    -- Modificaciones automáticas
    SET NEW.fecha_creacion = NOW();
    SET NEW.usuario_creacion = USER();
    SET NEW.campo_calculado = NEW.campo1 * NEW.campo2;

    -- Convertir a mayúsculas
    SET NEW.nombre = UPPER(NEW.nombre);

END //
```

DELIMITER ;

5. TRIGGER BEFORE UPDATE

```
DELIMITER //
```

```
CREATE TRIGGER trg_before_update_tabla
  BEFORE UPDATE ON tabla
  FOR EACH ROW
BEGIN
  -- Prevenir cambios en campos críticos
  IF OLD.campo_critico != NEW.campo_critico AND OLD.estado = 'BLOQUEADO' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'No se puede modificar un registro bloqueado';
  END IF;

  -- Actualizar campos de auditoría
  SET NEW.fecha_modificacion = NOW();
  SET NEW.usuario_modificacion = USER();

  -- Lógica condicional
  IF NEW.estado = 'ACTIVO' AND OLD.estado = 'INACTIVO' THEN
    SET NEW.fecha_activacion = NOW();
  END IF;

END //
```

```
DELIMITER ;
```

6. TRIGGER AFTER INSERT/UPDATE/DELETE

```
DELIMITER //
```

```
CREATE TRIGGER trg_after_insert_auditoria
  AFTER INSERT ON tabla_principal
  FOR EACH ROW
BEGIN
  INSERT INTO tabla_auditoria (
    tabla_afectada,
    operacion,
    id_registro,
    usuario,
    fecha
  ) VALUES (
    'tabla_principal',
    'INSERT',
    NEW.id,
```

```
        USER(),
        NOW()
    );
END //
```

```
CREATE TRIGGER trg_after_update_auditoria
    AFTER UPDATE ON tabla_principal
    FOR EACH ROW
BEGIN
    INSERT INTO tabla_auditoria (
        tabla_afectada,
        operacion,
        id_registro,
        valores_anteriores,
        valores_nuevos,
        usuario,
        fecha
    ) VALUES (
        'tabla_principal',
        'UPDATE',
        NEW.id,
        CONCAT('campo1:', OLD.campo1, ',campo2:', OLD.campo2),
        CONCAT('campo1:', NEW.campo1, ',campo2:', NEW.campo2),
        USER(),
        NOW()
    );
END //
```

```
CREATE TRIGGER trg_after_delete_auditoria
    AFTER DELETE ON tabla_principal
    FOR EACH ROW
BEGIN
    INSERT INTO tabla_auditoria (
        tabla_afectada,
        operacion,
        id_registro,
        valores_anteriores,
        usuario,
        fecha
    ) VALUES (
        'tabla_principal',
        'DELETE',
        OLD.id,
        CONCAT('campo1:', OLD.campo1, ',campo2:', OLD.campo2),
        USER(),
        NOW()
    );
END //
```

DELIMITER ;

PLANTILLAS DE PRUEBAS

Pruebas de Procedimientos

```
-- =====
-- PRUEBAS DE PROCEDIMIENTOS
-- =====

-- Prueba básica
CALL nombre_procedimiento(valor1, valor2);

-- Prueba con variables de salida
SET @resultado = 0;
CALL procedimiento_con_salida(100, @resultado);
SELECT @resultado AS resultado_obtenido;

-- Prueba de casos límite
CALL procedimiento(NULL); -- Valor nulo
CALL procedimiento(-1); -- Valor negativo
CALL procedimiento(999); -- Valor inexistente

-- Verificar estado antes y después
SELECT * FROM tabla WHERE condicion;
CALL procedimiento_modificador(parametros);
SELECT * FROM tabla WHERE condicion;
```

Pruebas de Triggers

```
-- =====
-- PRUEBAS DE TRIGGERS
-- =====

-- Mostrar estado inicial
SELECT 'ESTADO INICIAL:' AS info;
SELECT * FROM tabla WHERE id = 1;

-- Prueba INSERT
INSERT INTO tabla (campo1, campo2) VALUES ('valor1', 'valor2');
```

```
SELECT 'DESPUÉS DE INSERT:' AS info;
SELECT * FROM tabla WHERE campo1 = 'valor1';

-- Prueba UPDATE
UPDATE tabla SET campo1 = 'nuevo_valor' WHERE id = 1;
SELECT 'DESPUÉS DE UPDATE:' AS info;
SELECT * FROM tabla WHERE id = 1;

-- Prueba casos que deberían fallar
-- INSERT INTO tabla (campo1) VALUES (NULL); -- Debería dar error
-- UPDATE tabla SET campo_critico = 'cambio' WHERE estado = 'BLOQUEADO'; -- Debería

-- Verificar tabla de auditoría
SELECT * FROM tabla_auditoria ORDER BY fecha DESC LIMIT 5;
```

PATRONES COMUNES DE EXAMEN

1. Contadores y Estadísticas

```
-- Contar registros por categoría
SELECT COUNT(*) INTO v_contador
FROM tabla
WHERE categoria = p_categoria;

-- Calcular totales
SELECT SUM(campo_numerico) INTO v_total
FROM tabla
WHERE condicion;

-- Obtener máximos/mínimos
SELECT MAX(fecha) INTO v_fecha_maxima
FROM tabla;
```

2. Validaciones Típicas

```
-- Verificar existencia
SELECT COUNT(*) INTO v_existe
FROM tabla
WHERE id = p_id;
```



```
IF v_existe = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Registro no encontrado';
END IF;

-- Validar rangos
IF NEW.edad < 0 OR NEW.edad > 120 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Edad inválida';
END IF;

-- Validar formatos
IF NEW.email NOT LIKE '%@%.%' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Email inválido';
END IF;
```

3. Operaciones Jerárquicas (como familias)

```
-- Actualizar en cascada (padre -> hijos)
UPDATE tabla_hijos
SET campo = valor
WHERE padre_id = p_id_padre;

-- Buscar todos los descendientes
WITH RECURSIVE descendientes AS (
    SELECT id, nombre, padre_id
    FROM familias
    WHERE id = p_id_familia

    UNION ALL

    SELECT f.id, f.nombre, f.padre_id
    FROM familias f
    INNER JOIN descendientes d ON f.padre_id = d.id
)
SELECT * FROM descendientes;
```

4. Campos de Auditoría Automáticos

```
-- En triggers BEFORE INSERT
SET NEW.fecha_creacion = NOW();
SET NEW.usuario_creacion = USER();

-- En triggers BEFORE UPDATE
SET NEW.fecha_modificacion = NOW();
SET NEW.usuario_modificacion = USER();
```

-- Preservar fecha de creación

SET NEW.fecha_creacion = OLD.fecha_creacion;

SET NEW.usuario_creacion = OLD.usuario_creacion;

! ERRORES COMUNES A EVITAR

Sintaxis

- [] Olvidar DELIMITER // al inicio y ; al final
- [] No cerrar correctamente con END //
- [] Usar ; dentro del procedimiento sin cambiar el delimitador
- [] Declarar variables después de la lógica

Lógica

- [] No validar parámetros de entrada
- [] No manejar casos NULL
- [] Operaciones sin transacciones cuando se requieren
- [] No cerrar cursores abiertos

Triggers

- [] Crear loops infinitos (trigger que modifica la misma tabla)
- [] No distinguir entre OLD y NEW correctamente
- [] Usar SELECT dentro de triggers (mejor INSERT en tabla de log)

Buenas Prácticas

- [] Nombres descriptivos para variables y procedimientos
- [] Comentarios explicativos
- [] Manejo consistente de errores
- [] Pruebas exhaustivas con casos límite

CHECKLIST FINAL ANTES DE ENTREGAR

- ☐  Todos los procedimientos compilean sin errores
- ☐  Todos los triggers se crean correctamente
- ☐  Las pruebas funcionan como se espera
- ☐  Los casos de error lanzan mensajes apropiados
- ☐  El código está comentado y es legible
- ☐  Se incluyen tanto casos de éxito como de fallo
- ☐  Las transacciones se manejan correctamente
- ☐  No hay hardcoding de valores innecesario

CONSEJOS PARA EL EXAMEN

1. **Lee todo el enunciado** antes de empezar a programar
2. **Identifica las tablas** y sus relaciones
3. **Planifica la lógica** antes de escribir código
4. **Prueba cada componente** por separado
5. **Verifica los casos límite** (NULL, 0, valores inexistentes)
6. **Usa nombres descriptivos** para variables y procedimientos
7. **Comenta el código** para explicar la lógica compleja
8. **Incluye manejo de errores** apropiado
9. **Revisa la sintaxis** antes de ejecutar
10. **Documenta las pruebas** realizadas

¡Buena suerte en tu examen! 